

# Artigo: Criteria for Modularization

O artigo de D.L. Parnas, publicado em 1972 na revista Communications of the ACM, trata de um problema bem específico mas que acaba sendo fundamental: como dividir um sistema de software em módulos de forma inteligente. À primeira vista parece uma coisa técnica chata, mas na real o cara tá mexendo em algo bem mais profundo sobre como a gente pensa em estruturar código.

A motivação é simples. O Parnas começa lembrando que todo mundo sabe que modularizar é bom porque facilita o trabalho em equipe, permite fazer mudanças em uma parte sem quebrar tudo, e torna o sistema mais fácil de entender. Mas aí fica uma pergunta: se todo mundo concorda que modularizar é bom, por que tem tão pouca discussão sobre *como* fazer isso direito? E essa é a brecha que ele tenta preencher no artigo. Ele não quer ficar só discutindo teoria genérica, então pega um exemplo prático: um sistema de índice KWIC (que basicamente pega linhas de texto, faz rotações circulares nelas e ordena tudo alfabeticamente). É um problema pequeno o bastante para caber em um artigo, mas grande o bastante para mostrar o que ele quer argumentar.

Aí vem a parte interessante. O Parnas apresenta duas formas completamente diferentes de dividir esse mesmo sistema em módulos. A primeira forma é bem convencional: ele pensa no fluxo do processamento (entrada de dados, depois rotação circular, depois ordenação alfabética, depois saída) e cria um módulo para cada etapa. Isso faz sentido intuitivo e é o que a maioria dos programadores provavelmente faria. Parece lógico, segue a ordem das coisas.

A segunda forma é bem diferente. Ele pensa em quais são as decisões de design que podem mudar ou que são complicadas, e cria um módulo em volta de cada uma delas. Por exemplo, um módulo inteiro só fica responsável por guardar e acessar as linhas de texto, escondendo como essa informação é armazenada. Outro módulo cuida das rotações circulares, outro da ordenação. A diferença não é só na forma de contar os módulos, é na filosofia: ao invés de pensar no que o programa *faz*, ele pensa no que precisa ser *escondido*.

Quando ele compara os dois jeitos, as diferenças ficam meio óbvias em retrospecto. Na primeira abordagem, se você quiser mudar como os dados são armazenados em memória, praticamente todo módulo se afeta. Já na segunda, essa mudança fica confinada a um único módulo. O Parnas chama isso de "information hiding" (escondimento de informação) e é basicamente a ideia central do artigo.

Ele também fala de um problema bem prático: se você implementar a segunda abordagem de forma ingênua, com cada função sendo uma chamada de subrotina de verdade, tudo fica lento demais porque tem muito overhead de chamada de função. Mas aí ele sugere que pode não ser exatamente assim: pode usar assembler, pode usar técnicas especiais para fazer as coisas funcionarem rapidinho. Ou seja, a divisão lógica dos módulos pode ser completamente diferente da divisão física no código compilado. Isso foi bastante inovador pra época.

Um dos exemplos secundários que ele dá é interessante: ele mostra que a decomposição dele funcionou tanto para um compilador quanto para um interpretador da mesma linguagem, mesmo sendo sistemas bem diferentes. Isso mostra que a abordagem dele é mais robusta que pensar só em fluxo de execução.

Ele também discute estrutura hierárquica. Na segunda abordagem, alguns módulos dependem de outros, formando uma hierarquia. Isso significa que você consegue tirar partes do topo da árvore e ainda ter um sistema funcionando. É tipo: o módulo de armazenamento de dados pode ser usado em vários sistemas diferentes, enquanto a ordenação alfabética depende dele. Isso é útil pra reutilizar código.

No final, a conclusão é bem direta: não comece dividindo seu sistema por um fluxograma. Comece pensando em quais decisões de design são difíceis ou podem mudar, e faça cada módulo esconder uma dessas decisões. Isso vai deixar seu código mais flexível pra mudanças futuras e mais fácil de entender porque cada módulo tem uma responsabilidade clara.

O que eu acho legal desse artigo é que ele não é só teórico. O cara literalmente mostra duas decomposições diferentes do mesmo problema, descreve os trade-offs, e explica por que um jeito é melhor que o outro em alguns aspectos. É bem concreto.

A crítica que dá pra fazer é que em 1972 era mais prático isso porque os sistemas eram menores. Hoje em dia a gente tem outros desafios, tipo quando os módulos ficam muito grandes ou quando mexer em um módulo ainda afeta vários outros mesmo tentando esconder tudo. Mas a ideia central é sólida e influenciou bastante como a gente pensa em design de software desde então. É um daqueles papers antigos que a gente estuda em engenharia de software porque a lógica por trás continua válida.